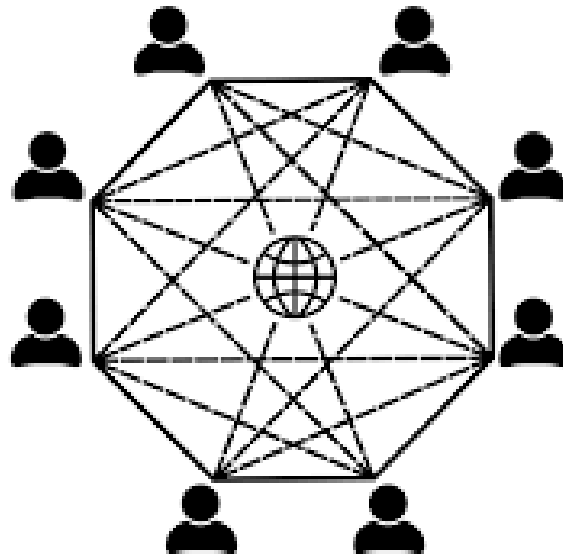
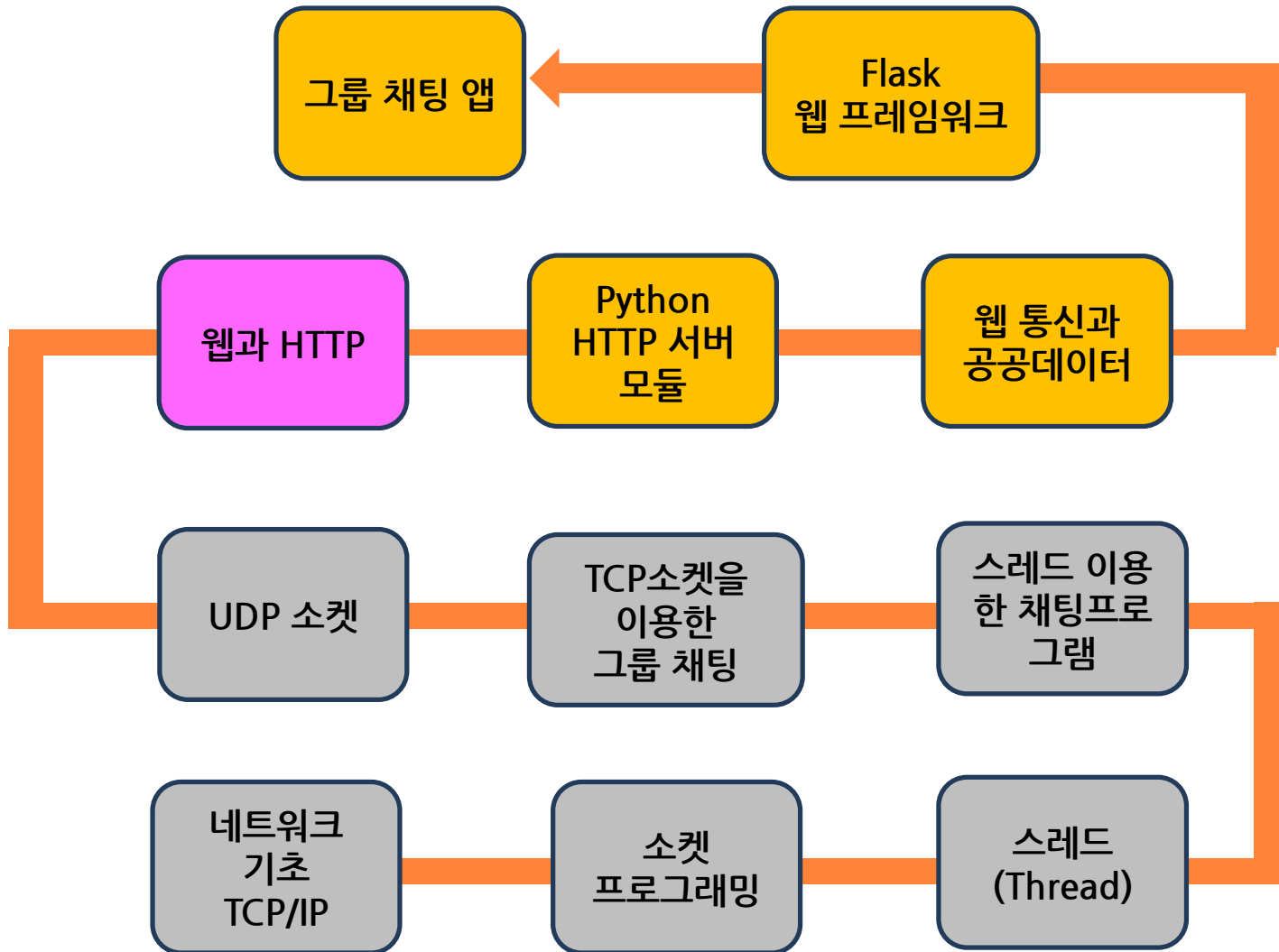


네트워크설계

9주차 - 웹과 HTTP



9주차부터 웹과 HTTP통신



앱 제작 실습 병행

1.학과소개 앱 제작하기

- ‘학과소개 앱’은 기존 6주차 map 앱에서 제작합니다
- 6주차에 제작한 ‘map(위치찾기) ’ 프로젝트에서 시작해 주세요
- 앱 인벤터 내 프로젝트에 map 프로젝트가 없으신 분은 CTL 사이트 공지 사항에서 다운받아 사용하세요

“ 학과소개 앱 실습에는 iOS가 지원하지 않는 컴포넌트가 사용됩니다.
아이폰 사용자는 실습용 안드로이드 폰을 사용해 앱을 검증해 주세요!”

1.학과소개 앱 제작하기

■ 학과소개 앱 제작하기

이번에 알아볼 앱 인벤터 제작 방법은

1) 멀티 스크린 사용하기

한 프로젝트에서 앱 스크린을 여러 개 만들어 봅시다.
4주차에 제작한 '위치찾기 앱'에 추가하여 제작하도록
하겠습니다 (map앱에서 추가 제작)

2) [목록뷰] 사용하기 : 일정관리 앱에서 사용

[목록뷰] 컴포넌트는 스마트폰 화면에 목록을 보여주는
컴포넌트입니다. 여러 항목들을 목록형식으로 스마트폰 화면
에 표시해주어 사용자가 원하는 목록을 선택하고
그 목록의 정보를 제공해 주는 앱에서 많이 사용하는
방식입니다

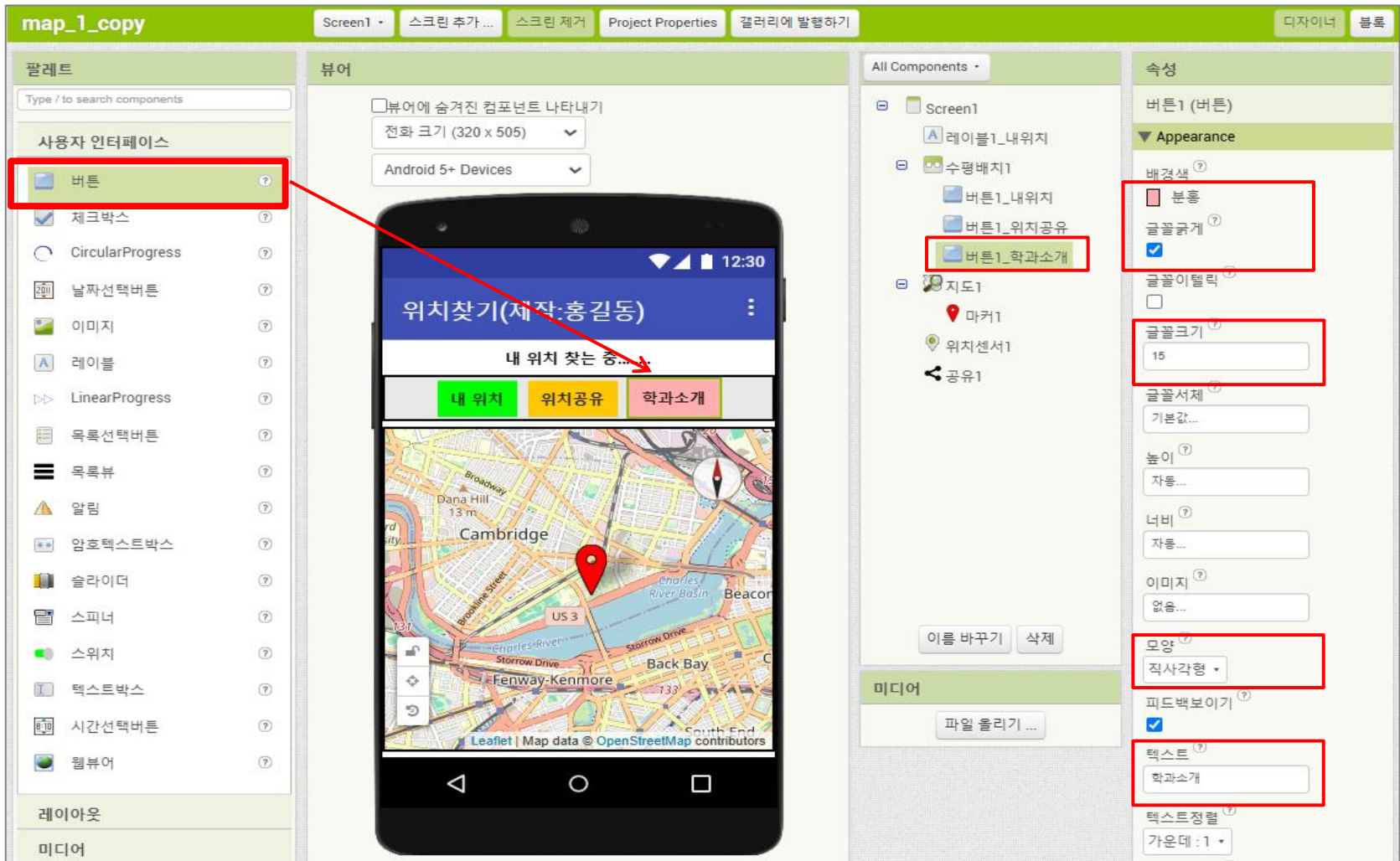
멀티스크린



1.학과소개 앱 제작하기 - 스크린 추가하기

1. Screen1 디자인하기 (※ 주의: 6주차 제작한 map 프로젝트에서 추가작업)

- [버튼] 추가 (이름: 버튼1_학과소개, 속성>> 배경-분홍, 글꼴굵게, 글꼴크기-15, 모양-직사각형, 텍스트- 학과소개)



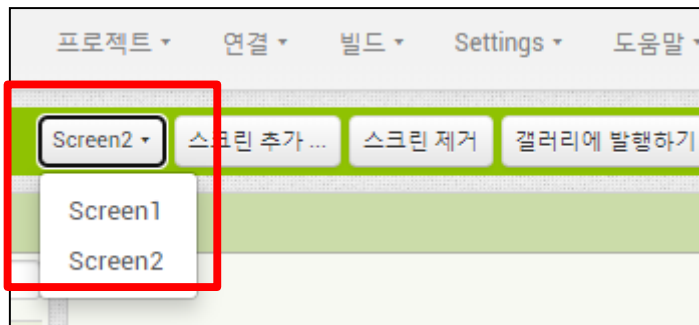
1.학과소개 앱 제작하기 - 스크린 추가하기

1. Screen1 디자인하기

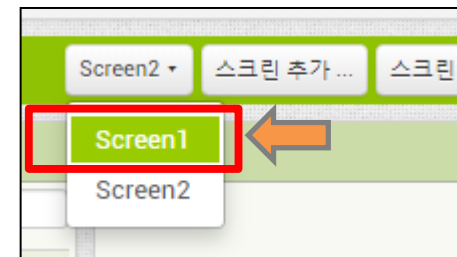
① Screen2 추가하기



② 추가하면 자동적으로 Screen2 뷰어창으로 이동



③ 다시 Screen1창으로 이동



1.학과소개 앱 제작하기 - 스크린 추가하기

2. Screen1 코딩하기 (※ 주의: Screen1에서 작업)

- Screen1에서 Screen2로 이동하는 기능 추가하기

The screenshot illustrates the process of adding a navigation function to Screen1. On the left, the '공통 블록' (Common Blocks) panel shows the '제어' (Control) block selected with a red dashed box. Below it, the 'Screen1' panel shows the '버튼1_학과소개' (Button1_Subject Introduction) block also selected with a red dashed box. A red arrow points from the '제어' block to a logic block in the center. This logic block contains the following code: '언제 버튼1_학과소개 .클릭했을때' (When Button1_Subject Introduction is clicked), '실행' (Execute), '다른 스크린 열기' (Open another screen), and '스크린 이름' (Screen name) set to 'Screen2'. To the right, a dropdown menu shows 'Screen1' selected with a checkmark, and 'Screen2' is listed below it. A red arrow points from the 'Screen2' option in the dropdown to the '스크린 이름' field in the logic block. Below the dropdown, a note states: '※ Screen2를 추가하면 아래와 같이 Screen2가 보입니다. Screen2 선택' (※ When you add Screen2, it will look like this. Select Screen2).

공통 블록

- 제어
- 논리
- 수학
- 텍스트
- 리스트

Screen1

- 레이블1_내위치
- 수평배치1
 - 버튼1_내위치
 - 버튼1_위치공유
 - 버튼1_학과소개
- 지도1
 - 마커1
 - 위치센서1

언제 버튼1_학과소개 .클릭했을때

실행

다른 스크린 열기 스크린 이름

Screen2

✓ Screen1

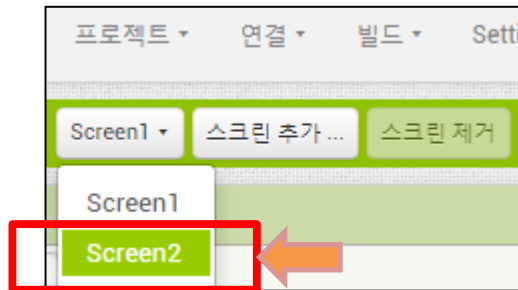
Screen2

※ Screen2를 추가하면 아래와 같이 Screen2가 보입니다. Screen2 선택

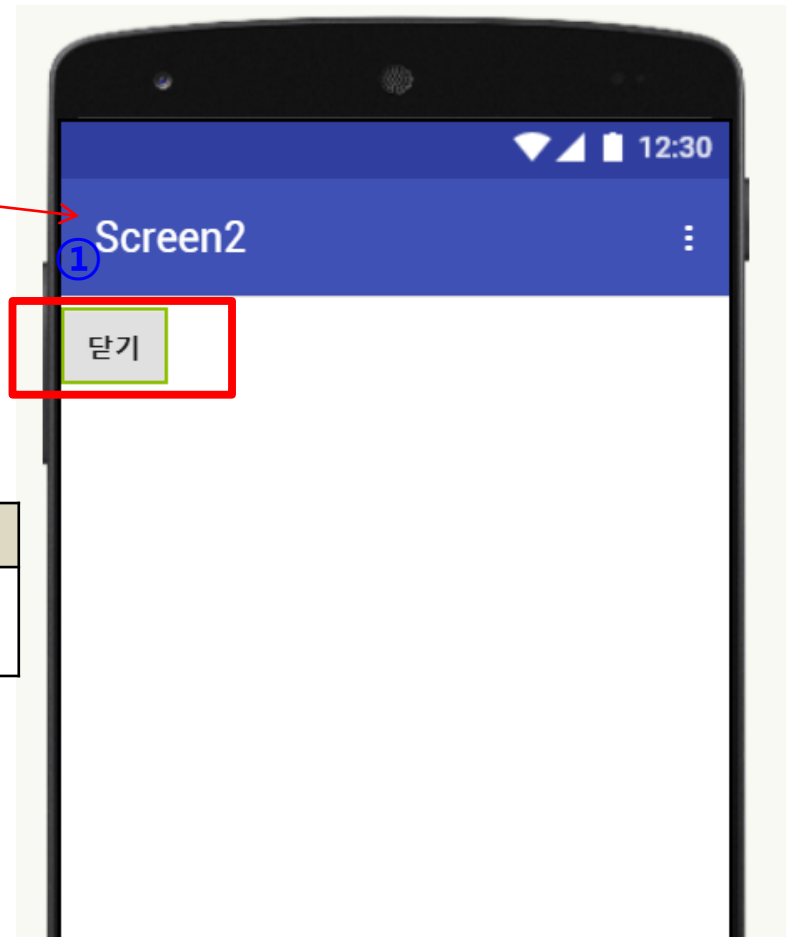
1.학과소개 앱 제작하기 - 스크린 추가하기

3. Screen2 디자인하기

- Screen2로 이동하기 : Screen2의 새로운 뷰어창이 나타납니다



※ Screen2에서 작업



[버튼] 디자인

컴포넌트	이름	속성
버튼	버튼1_닫기	글꼴굵게, 글꼴크기-15 모양-직사각형, 텍스트 - 닫기

1.학과소개 앱 제작하기 - 스크린 추가하기

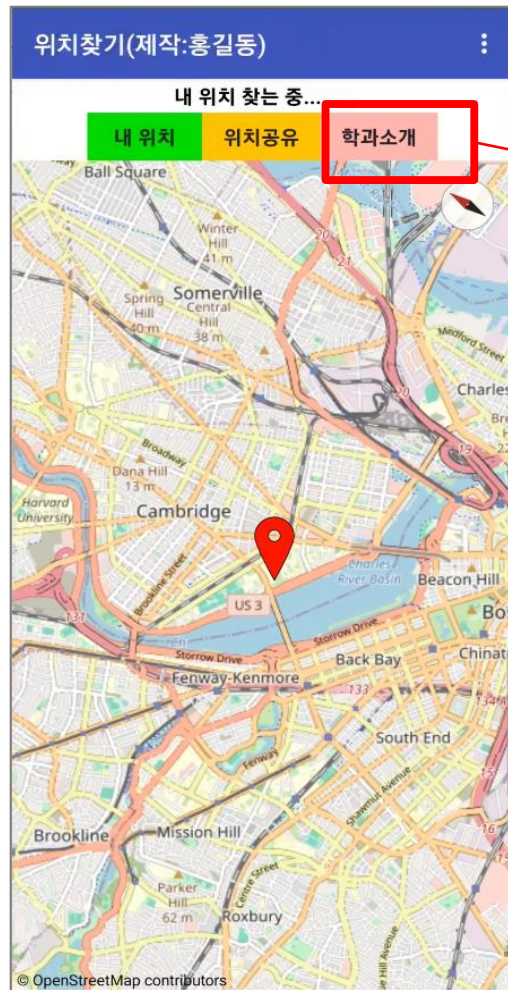
4. Screen2 앱 코딩하기

[닫기] 버튼을 클릭하면 'screen2' 화면을 닫고 'screen1'으로 이동하기



1.학과소개 앱 제작하기 - 스크린 추가하기

5. Screen 이동하기 완성



Screen2로 이동

Screen2를 닫고
Screen1으로 이동



1.학과소개 앱 제작하기 - 앱 디자인

■ 새로 추가한 Screen2에 '학과소개 앱' 제작하기

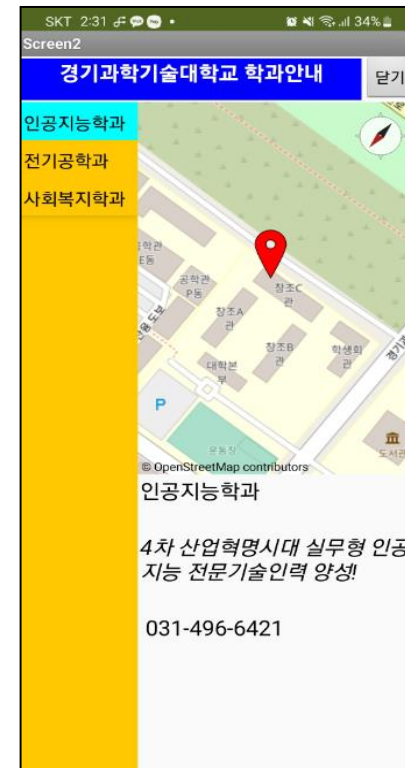
“ 앱 인벤터의 [목록뷰]를 사용하여 스마트폰 화면에 목록을 보여주고,
사용자가 선택한 목록의 상세 정보를 나타내는 기능을 제작해 봅시다”

사용 예) 주변 맛집, 약국등



그림출처: 헬로소프트

제작할 앱은
우리대학의 학과소개 앱



1.학과소개 앱 제작하기 - 앱 디자인

■ 구글맵에서 원하는 장소의 위도와 경도 알아내기

“구글지도에서 ‘경기과학기술대학교’ 검색”

공유

링크 보내기 지도 퍼가기

37.338664, 126.736138
경기도 시흥시 경기과학기술대학교

공유할 링크
<https://maps.app.goo.gl/pyhXaeWsHG3H6b3q9> 링크 복사

위도, 경도
알 수 있음

이 위치 공유

여기를 출발지로 설정
여기를 목적지로 설정
이곳이 궁금한가요?
주변검색
인쇄
누락된 장소 추가
비즈니스 추가
데이터 문제 신고
거리 측정

마우스 오른쪽 클릭

경기과학기술대학교

3.9 ★★★★★ (148)
전문대학 · ♿

개요 리뷰 정보

경로 저장 주변 휴대전화로 보내기 공유

경기도 시흥시 경기과학기술대 269

qttec.ac.kr

레이어

1.학과소개 앱 제작하기 - 앱 디자인

1. 학과소개 앱 디자인하기

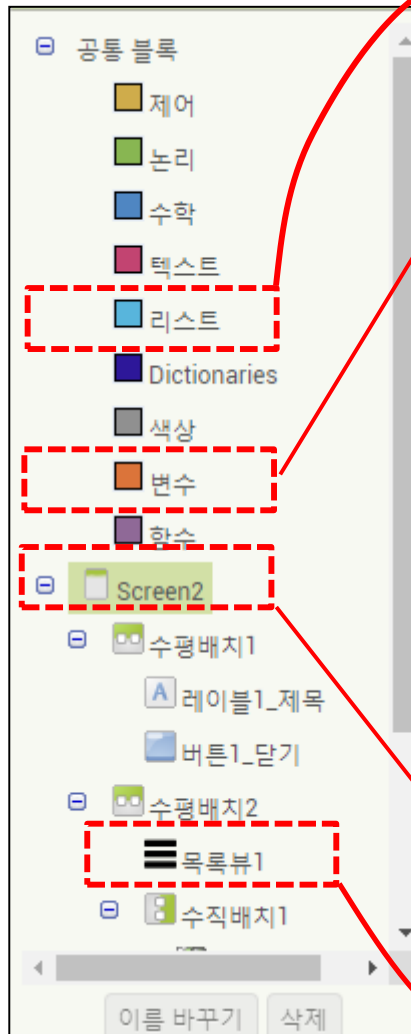
※ Screen2에서 작업



	컴포넌트	이름	속성
①	수평배치1	수평배치1	수평,수직정렬 - 가운데, 너비-부모요소
	레이블1	레이블1_제목	배경색-청록색, 글꼴크기-15 글꼴굵게, 높이와 너비 - 부모요소 텍스트- 경기과학기술대학교 학과소개 텍스트정렬 - 가운데
	버튼1	버튼1_닫기	텍스트- 닫기
②	수평배치2	수평배치2	높이 - 부모요소, 너비-부모요소
③	목록뷰1	목록뷰1	배경색-주황, 높이- 부모요소, 너비 - 30퍼센트 선택된항목색상 - 청록색 텍스트색상 - 검정, 텍스트크기 - 15
④	수직배치1	수직배치1	높이 - 부모요소, 너비 - 부모요소
⑤	지도1	지도1	중심좌표- 37.338653, 126.736127 높이-50 퍼센트, 너비- 부모요소 나침반보이기, 줌레벨 - 17
⑥	마커1	마커1	위도- 37.338653, 경도- 126.736127
⑦	레이블1	레이블1_학과이름	글꼴굵게, 글꼴크기-20, 텍스트- 학과이름, 텍스트색상 - 파랑
⑧	레이블1	레이블1_학과소개	글꼴크기-20, 텍스트- 학과소개
⑨	레이블1	레이블1_학과연락처	글꼴크기-20, 텍스트- 학과연락처

1.학과소개 앱 제작하기 - 앱 코딩

2. 학과소개 앱 코딩하기



전역변수 만들기 학과이름 초기값 리스트 만들기

인공지능학과
전기공학과
사회복지학과

학과이름 = ["인공지능학과", "자동차과", "사회복지과"]

전역변수 만들기 학과소개 초기값 리스트 만들기

4차 산업혁명시대 실무형 인공지능 전문기술인력 양성!
전기기술, 스마트 팩토리 분야의 중견기술인력 양성!
사회복지현장에서 성실하게 일하는 전문적 지식, 기술, 가치, 그리고 인성을 갖춘 사회복지...

전역변수 만들기 학과연락처 초기값 리스트 만들기

031-496-6421
031-496-4631
031-496-4665

※ 학과소개와 학과연락처는 학과 홈페이지에서 가져옵니다

전역변수 만들기 위도 초기값 리스트 만들기

37.339467
37.337779
37.337437

전역변수 만들기 경도 초기값 리스트 만들기

126.736326
126.738203
126.736678

※ 학과의 위도와 경도 정보는 구글지도에서 가져 옵시다

- 인공지능학과: 창조C관
- 전기공학과: 자동차관
- 사회복지과: 창조E관

언제 Screen2 초기화되었을때
실행 지정하기 목록뷰1 요소 값 가져오기 전역변수 학과이름

1.학과소개 앱 제작하기 - 앱 코딩

2. 학과소개 앱 코딩하기 (계속)

The image shows a mobile app development interface with a storyboard. On the left, a panel lists available UI elements: 리스트 (List), Dictionaries, 색상 (Color), 변수 (Variable), 함수 (Function), Screen2, 수평배치1 (Horizontal Layout 1), 레이블1_제목 (Label 1 Title), 버튼1_닫기 (Button 1 Close), 수평배치2 (Horizontal Layout 2), 목록뷰1 (List View 1), 수직배치1 (Vertical Layout 1), 지도1 (Map 1), 마커1 (Marker 1), 레이블1_학과이름 (Label 1 Department Name), and 레이블1_학과연락처 (Label 1 Department Contact). Red dashed boxes highlight '리스트' (List), '목록뷰1' (List View 1), '수직배치1' (Vertical Layout 1), '지도1' (Map 1), and '마커1' (Marker 1). Red arrows point from these elements to code blocks in the storyboard. The storyboard has a background color of '목록뷰1' (List View 1) and a title of '선택후에' (After Selection). The code blocks are organized into three main sections: 1. List View 1 (목록뷰1) - 선택후에 (After Selection): This section contains three '지정하기' (Specify) blocks for '레이블1_학과이름' (Label 1 Department Name), '레이블1_학과소개' (Label 1 Department Introduction), and '레이블1_학과연락처' (Label 1 Department Contact). Each '지정하기' block is connected to a '가져오기' (Fetch) block for '목록뷰1' (List View 1) and '선택된항목' (Selected Item). The '가져오기' blocks are connected to '전역변수' (Global Variable) blocks for '학과소개' (Department Introduction), '학과연락처' (Department Contact), and '위도' (Latitude). 2. Map View 1 (지도1) - 팬하기 (Pan): This section contains three '가져오기' (Fetch) blocks for '위도' (Latitude), '경도' (Longitude), and '주소' (Address). Each '가져오기' block is connected to a '전역변수' (Global Variable) block for '위도' (Latitude), '경도' (Longitude), and '주소' (Address). 3. Map View 1 (지도1) - 줌 (Zoom): This section contains three '가져오기' (Fetch) blocks for '위도' (Latitude), '경도' (Longitude), and '주소' (Address). Each '가져오기' block is connected to a '전역변수' (Global Variable) block for '위도' (Latitude), '경도' (Longitude), and '주소' (Address). A callout box on the right explains the variable naming convention: '※ 주의!! 선택된항목 (X) 선택된항목번호 (O)' (Note!! Selected Item (X) Selected Item Number (O)).

※ 주의!!
선택된항목 (X)
선택된항목번호 (O)

1.학과소개 앱 제작하기 - 앱 실행

3. 학과소개 앱 실행하기

- 학과를 목록에서 선택하면
학과 위치가 지도에 표시되며, 지도 밑에 학과이름, 학과소개, 학과 연락처가 출력됨



2. HTTP(HyperText Transfer Protocol) 개요

■ 웹(Web) 개요

- 인터넷을 이용해 할 수 있는 기능 중의 하나 (인터넷 >> 웹)
- 멀리 떨어져 있는 컴퓨터간 문서를 주고 받기 위해 탄생 (1990년)
- 웹의 4가지 구성요소
 - ① 웹 브라우저와 웹 서버 - 웹 페이지를 주고받는 소프트웨어
 - ② HTML - 웹 페이지를 만드는 언어
 - ③ URL - 원하는 웹 페이지가 어디에 있는지 알려주는 주소체계
 - ④ HTTP - 웹 브라우저와 웹 서버가 통신할 때 사용하는 통신규칙

2. HTTP(HyperText Transfer Protocol) 개요

■ URL Uniform Resource Locator

- 웹 주소, 네트워크 상에서 리소스가 어디 있는지 알려주기 위한 주소체계
- URL 형식: **프로토콜 // IP 주소 또는 도메인 주소 : 포트번호 / 자원 위치**

* 아래는 네이버 지도에서 '경기과학기술대학교' 검색시의 URL이다

[https:// map.naver.com \(:443\) /p/search/경기과학기술대학교 ?c=15.00,0,0,2,dh](https://map.naver.com(:443)/p/search/경기과학기술대학교?c=15.00,0,0,2,dh)
Protocol host port path query

- ① Protocol(프로토콜): 어떤 프로토콜을 사용하는지를 알려주는 역할 (http 또는 https)
- ② Host(호스트): 해당 웹사이트의 도메인 주소(네임 서버)
- ③ Port(포트) : URL엔 생략되어 있음 (http는 80, https는 443을 기본포트)
- ④ Path(경로): 해당 웹사이트의 더 자세한 경로를 나타내는 역할
- ⑤ Query(쿼리) : URL에서 "?" 뒤에 조건에 맞는 파라미터가 따라오며
입력한 값에 따라 다른 결과를 보여줘야 할 때 쿼리 문자열을 사용

2. HTTP(HyperText Transfer Protocol) 개요

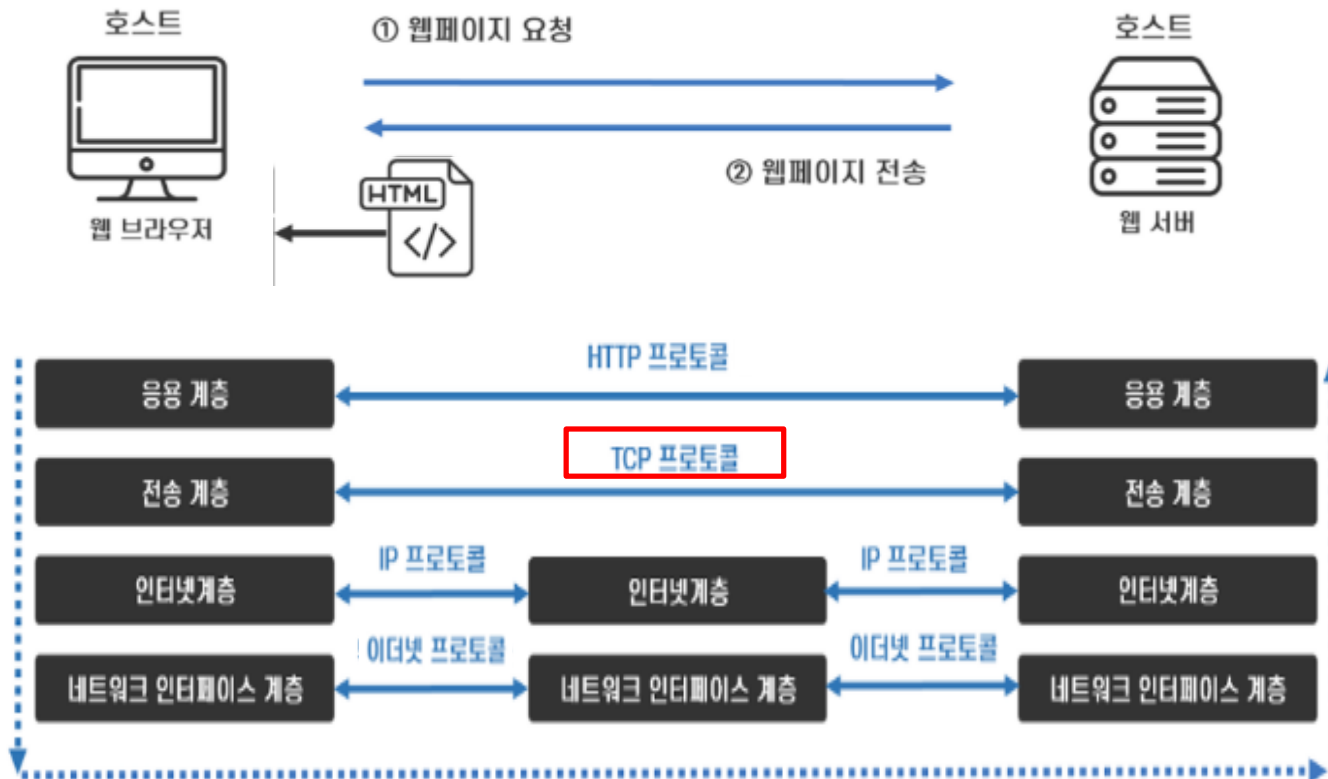
■ HTTP 개요

- 서버와 클라이언트간 HTML 문서를 주고 받기 위한 통신 규칙으로

웹 브라우저의 요청(Request) 메시지와 웹 서버의 응답(Response) 메시지의 구조를 규정

- HTTP (HyperText Transfer Protocol)과

HTTPS (HyperText Transfer Protocol over Secure Socket Layer): 보안이 강화된 HTTP

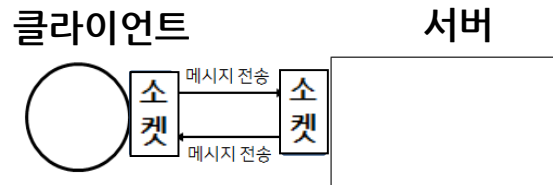


2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP 개요

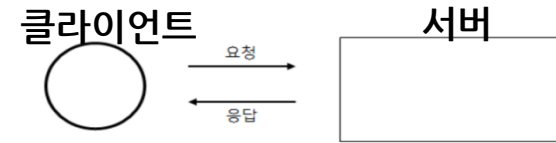
◦ HTTP 통신과 소켓 통신의 차이

소켓 통신



- 서버와 클라이언트 양방향 통신
:클라이언트와 서버 양쪽에서 서로에게 데이터를 전달하는 방식의 양방향 통신
- 서버나 클라이언트에서 강제로 접속을 해제할 때 까지는 계속 연결을 유지하는 방식
: 클라이언트 수가 늘어날 수록 서버의 부하 증가 → 접속할 수 있는 클라이언트 수 제한

HTTP 통신



- 클라이언트 요청이 있을 때 서버가 응답하는 방식. 단방향 통신
:클라이언트에서 서버로 요청을 보내고 서버는 응답하는 방식
- JSON, Image, HTML 파일 등 다양한 파일을 전송 받을 수 있음
- 응답을 받은 후 연결이 끊어지는 것이 기본 동작(HTTP 1.0)
- 성능 상으로 필요하다면 Keep Alive 옵션을 주어 일정 시간 동안 연결을 유지하는 것이 가능 (HTTP 1.1 이상)

“HTTP 통신은 소켓통신의 일종”

2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP 개요

○ HTTP 특징

① 서버와 클라이언트 구조

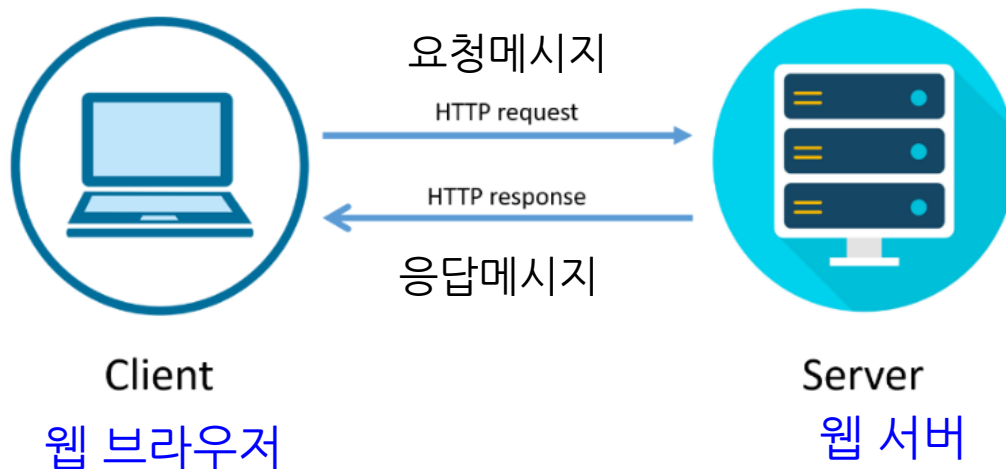
② 무상태 프로토콜(Stateless)

- 서버는 클라이언트 요청정보에 대해서만 응답하며, 이전 요청과는 무관

③ 비연결성 (Connectionless) 프로토콜

- HTTP는 기본적으로 연결을 유지하지 않는 모델
단, HTTP 1.1부터 HTTP 지속 연결 가능 (Persistent Connections)

<https://comic.naver.com/index>



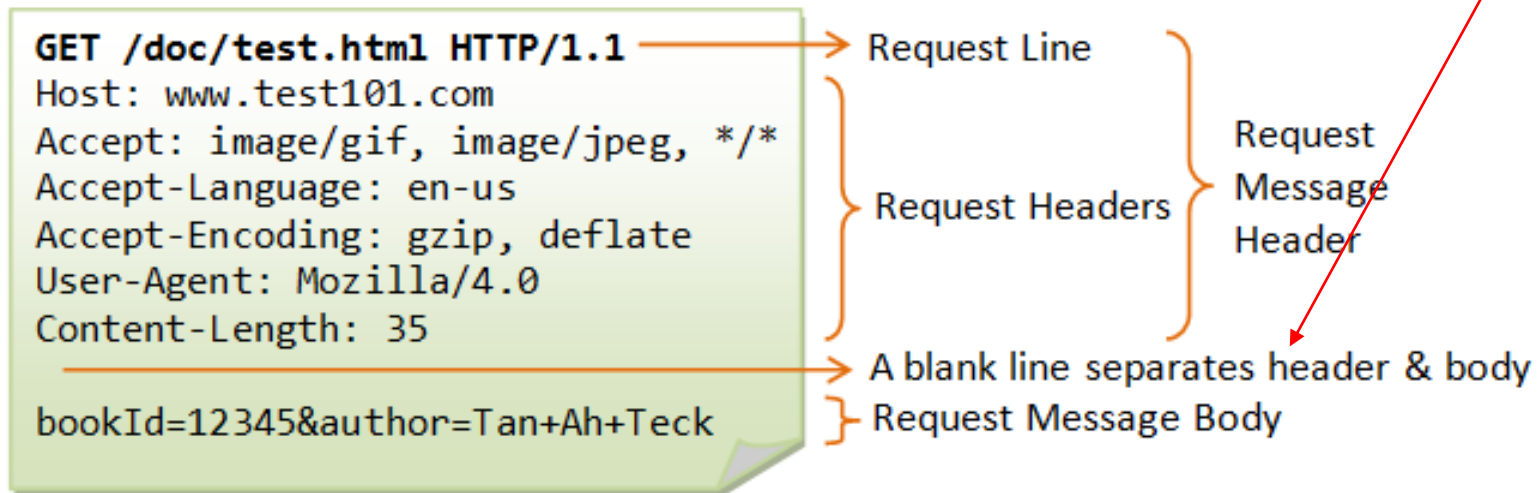
2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP 요청 메시지 (Request Message) 구조

○ HTTP Request Message의 구성

- Request Line
- Headers
- blank line (공백)
- Body

헤더와 바디를 구분하기 위해
헤더와 바디 사이에
blank line(공백)이 있음
→ **CRLF(\r\n)** : 공백라인



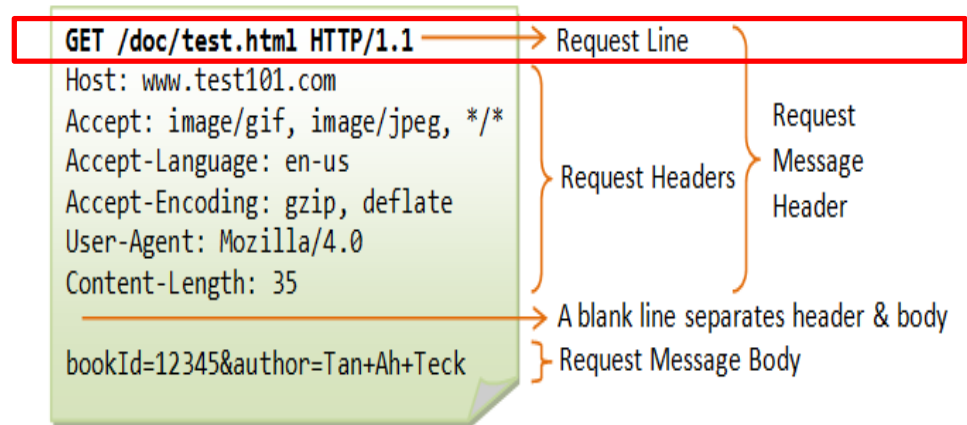
그림출처: gnaseel.tistory.com

2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP Request Message 구조

Request line

- HTTP Request Message의 시작 라인
- 3가지 부분으로 구성
 - ① HTTP method
 - ② URL
 - ③ HTTP version



```
GET /doc/test.html HTTP/1.1
[HTTP Method] [URL] [HTTP version]
```

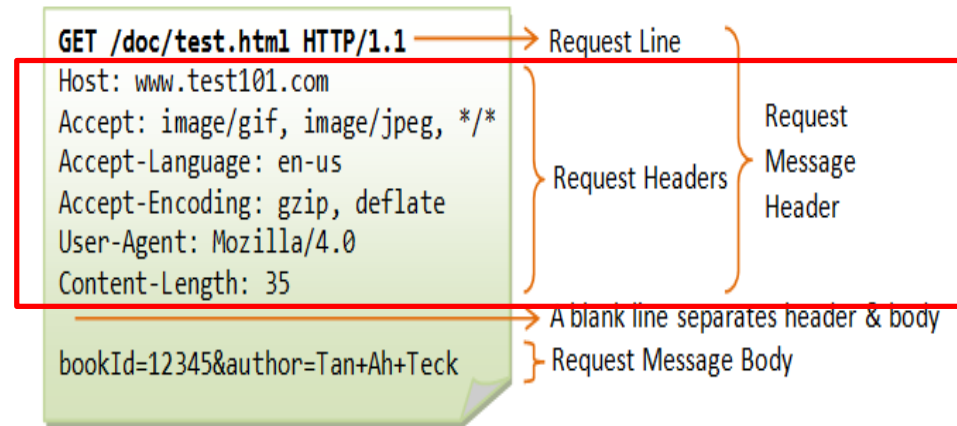
- ① HTTP Method : 클라이언트가 웹 서버에게 요청하는 목적을 알리는 수단
종류) GET 방식, POST 방식등
 - GET은 서버로 정보를 요청하는 목적 (“서버야 ~~ 좀 줘!”)
 - POST는 서버로 정보를 전달하는 목적 (“서버야 ~~ 줄 테니 처리해 줘”)
- ② URL : 요청 자원 명시
- ③ HTTP version: HTTP version을 명시(HTTP 1.0, 1.1, 2.0 등)

2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP Request Message 구조

Headers

- 해당 request에 대한 추가 정보를 담고 있는 부분



HTTP 요청 메시지에서 사용하는 헤더 항목

* *Header*는 *Request Line*에서 표현되지 않은 더 구체적인 요구(추가정보)를 작성하는 공간

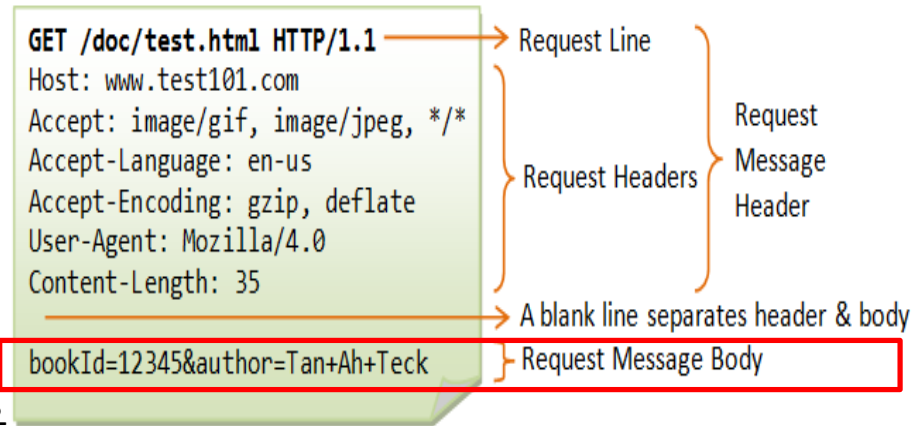
- **Host** : 서버의 도메인 주소
- **Connection** : Keep-alive가 디폴트 (HTTP/2에선 사용하지 않음)
- **User-Agent** : 사용자의 웹 브라우저 종류&버전 정보
- **Accept** : 브라우저가 처리할 수 있는 데이터의 형태
- **Accept-Language** : 사용하는 언어
- **Accept-Encoding** : 브라우저가 처리할 수 있는 콘텐츠 압축 방식

2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP Request Message 구조

Body

- HTTP Request가 전송하는 데이터를 담고 있는 부분
- 전송하는 데이터가 없다면 body 부분은 비어있음 즉, GET 요청 시에는 Body는 비어있고 POST 요청일 경우 Body에 정보가 포함되어 있음



2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP 응답메시지(Response Message) 구조

○ HTTP Response Message의 구성

- Status Line
- Headers
- Blank line(공백)
- Body

헤더와 바디를 구분하기 위해
헤더와 바디 사이에
blank line(공백)이 있음
→ **CRLF(\r\n)** : 공백라인

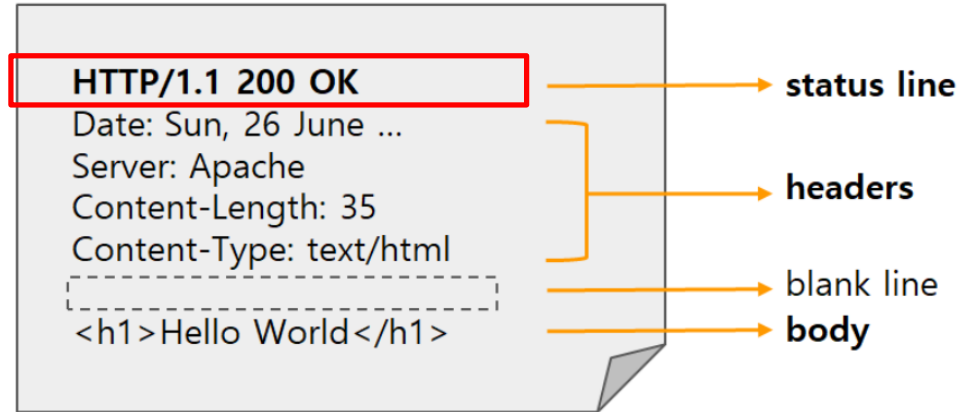


2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP Response Message 구조

Status line

- HTTP Response의 상태를 나타내는 부분
- 3가지 부분으로 구성
 - ① HTTP version
 - ② Status Code
 - ③ Status Text



HTTP/1.1 200 OK
[HTTP version] [Status Code] [Status Text]

HTTP Status Codes

Level 200

200: OK
201: Created
202: Accepted
203: Non-Authoritative Information
204: No content

Level 400

400: Bad Request
401: Unauthorized
403: Forbidden
404: Not Found
409: Conflict

Level 500

500: Internal Server Error
501: Not Implemented
502: Bad Gateway
503: Service Unavailable
504: Gateway Timeout
599: Network Timeout

3xx Redirection

301	Moved Permanently
302	Found
303	See Other
304	Not Modified

2. HTTP(HyperText Transfer Protocol) 개요

■ HTTP Response Message 구조

Headers

- Response의 headers와 거의 유사

- 네트워크상에서 데이터를 보낼 때 데이터의 형식을 나타내는 표준방식인 MIME 타입 사용

**MIME* *Multipurpose Internet Mail Extensions* : 미디어 타입 (주타입/서브타입)

서버는 모든 HTTP 객체에 MIME 타입을 붙임

- **Content-type**: 보내는 자원의 형식을 명시하기 위해 헤더에 실리는 정보 (MIME 타입)

예) Content-Type: text/html : text 중 html 파일임을 나타냄

- **Content-Length** : body의 길이(바이트 단위)



Body

- 보통은 클라이언트가 요청한 HTML 문서가 들어감

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 실습1) 웹서버와 웹 브라우저 통신 실습

- 간단한 HTML 문서 작성
*메모장이나 파이썬 개발환경에서 작성

```
<html>

<body>

    <h1>Simple HTTP Server</h1>

    <p>This is index page</p>

</body>

</html>
```

→ 파일 저장 시
index.html 로 저장 (확장자 주의)

```
<html>

<body>

    <h1>Simple HTTP Server</h1>

    <p>This is hello page</p>

</body>

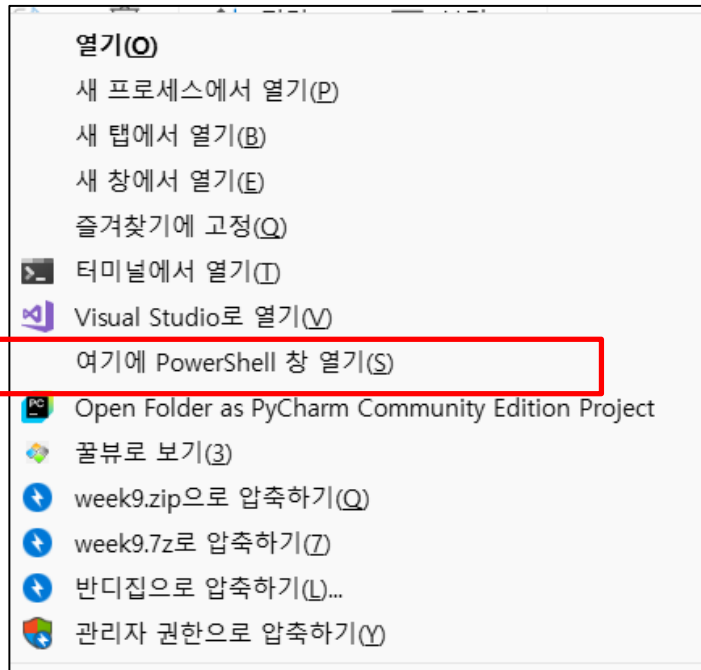
</html>
```

→ 파일 저장 시
hello.html 로 저장 (확장자 주의)

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 실습1) 웹 서버와 웹 브라우저 통신 실습

○ 파이썬 명령어로 로컬 서버 실행하기



① Index.html 문서가 있는 폴더에서
Shift + 마우스 오른쪽 클릭

② **여기에 PowerShell 창 열기** 클릭

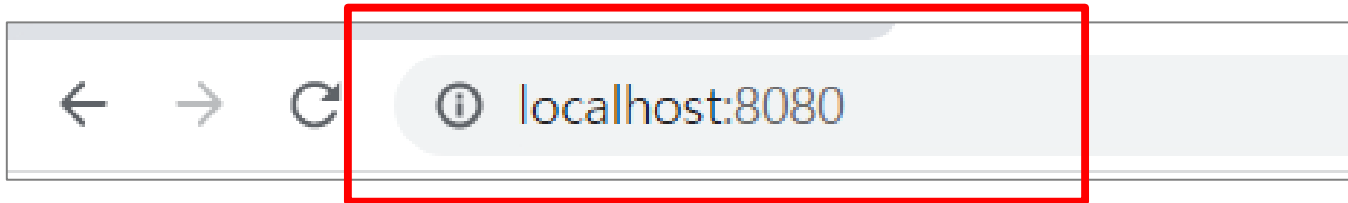
③ **python -m http.server 8080** 실행

```
Windows PowerShell
PS C:\Jupyter_Project\NetworkDesign\week9> python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
```

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 실습1) 웹서버와 웹 브라우저 통신 실습

- 웹 브라우저 실행
 - 주소창에 localhost:8080 입력 후 실행



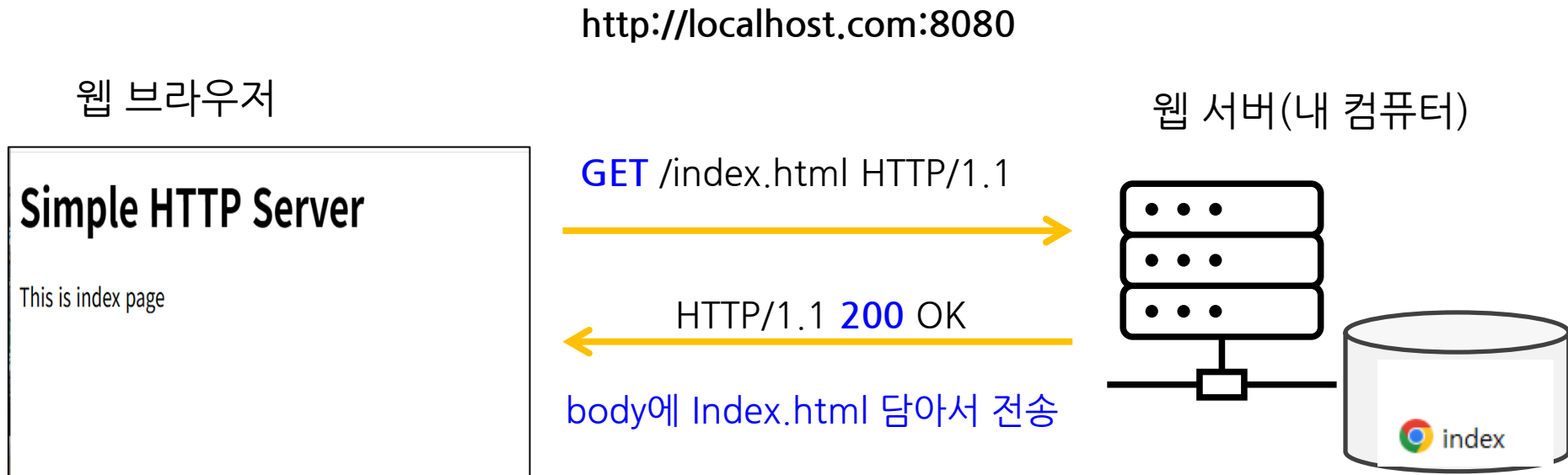
<http://localhost.com:8080/index.html>



URL의미) 서버(내 컴퓨터)있는 index.html 파일을 요청하는 주소

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 실습1) 웹서버와 웹 브라우저 통신 실습



```
PS C:\Jupyter_Project\NetworkDesign\week9> python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::1 - - [27/Apr/2025 15:15:35] "GET / HTTP/1.1" 200 -
```

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 개발자도구(F12)로 확인하기

- 개발자도구 : 서버와 클라이언트간 주고받는 데이터 확인하는 기능
- 크롬 웹 브라우저에서 단축키 F12를 누르면 표시

Simple HTTP Server

This is index page

The screenshot shows the Chrome DevTools Network tab. A request to `http://localhost:8080/` is selected. The **Headers** sub-tab is active, showing the **General** section with the following details:

- Request URL: `http://localhost:8080/`
- Request Method: **GET** (highlighted with a red box)
- Status Code: **200 OK** (highlighted with a red box)
- Remote Address: `[::1]:8080`
- Referrer Policy: `strict-origin-when-cross-origin`

Below the General section, the **Response Headers** section is visible, with the **Raw** checkbox checked. The response headers are:

- Content-Length:** 94 (highlighted with a red box)
- Content-Type:** `text/html` (highlighted with a red box)
- Date:** Sun, 27 Apr 2025 06:16:00 GMT
- Last-Modified:** Sun, 27 Apr 2025 06:09:06 GMT
- Server:** SimpleHTTP/0.6 Python/3.12.7

▼ Response Headers

☒ Raw

HTTP/1.0 200 OK

Server: SimpleHTTP/0.6 Python/3.12.7

Date: Sun, 27 Apr 2025 06:16:00 GMT

Content-type: text/html

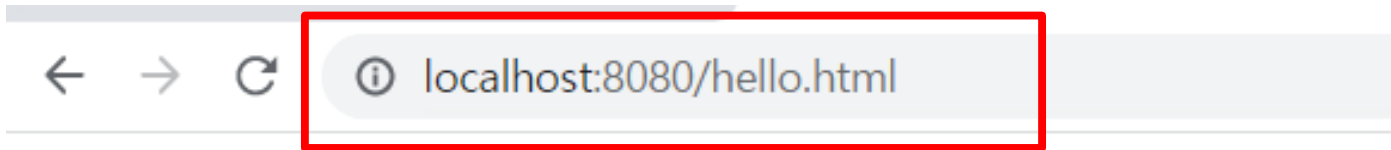
Content-Length: 94

Last-Modified: Sun, 27 Apr 2025 06:09:06 GMT

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 실습1) 웹서버와 웹 브라우저 통신 실습

- 웹 브라우저 실행
 - 주소창에 localhost:8080/hello.html 입력 후 실행



<http://localhost.com:8080/hello.html>



URL의미) 서버(내 컴퓨터)에 있는 hello.html 파일을 요청하는 주소

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

- 개발자도구(F12)로 확인하기

← → ↻ ⓘ localhost:8080/hello.html

Simple HTTP Server

This is hello page

The screenshot shows the Chrome DevTools Network tab. The 'hello.html' request is selected. The 'Headers' sub-tab is active, showing the 'General' section. The 'Request Method' is 'GET' and the 'Status Code' is '200 OK', both highlighted with red boxes. The 'Response Headers' section is also visible, with 'Content-Length: 94' and 'Content-Type: text/html' highlighted by a red box.

Name	Value
Request URL:	http://localhost:8080/hello.html
Request Method:	GET
Status Code:	200 OK
Remote Address:	[::1]:8080
Referrer Policy:	strict-origin-when-cross-origin

Response Headers	Value
Content-Length:	94
Content-Type:	text/html
Date:	Sun, 27 Apr 2025 06:23:47 GMT
Last-Modified:	Sun, 27 Apr 2025 06:07:11 GMT
Server:	SimpleHTTP/0.6 Python/3.12.7

3. 웹 서버와 웹 브라우저간 HTTP 통신 실습

■ 실습1) 웹서버와 웹 브라우저 통신 실습

웹 브라우저

http://localhost.com:8080



GET /index.html HTTP/1.1



HTTP/1.1 200 OK

body에 Index.html 담아서 전송

http://localhost.com:8080/hello.html



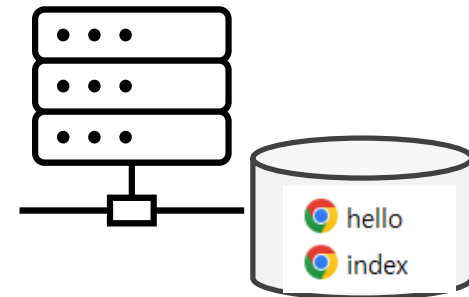
GET heool.html HTTP/1.1



HTTP/1.1 200 OK

body에 hello.html 담아서 전송

웹 서버(내 컴퓨터)



```
PS C:\Jupyter_Project\NetworkDesign\week9> python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::1 - - [27/Apr/2025 15:15:35] "GET / HTTP/1.1" 200 -
::1 - - [27/Apr/2025 15:23:47] "GET /hello.html HTTP/1.1" 200 -
```

4. 파이썬으로 간단한 HTTP 서버 제작하기

- 실습2) 간단한 HTTP 서버를 제작하여 웹 브라우저와 연동하기



Simple HTTP 서버
([simpleHttpServer.py](#))



Response message 구조

```
HTTP/1.1 200 OK
Content-length: body의 길이(바이트)
Content-type: text/html

body
```

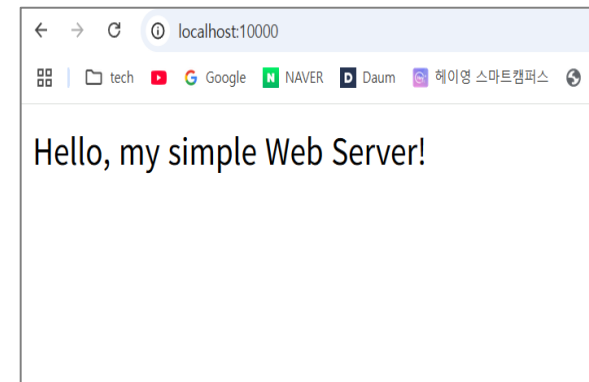
GET

← HTTP Requests

200

→ HTTP Response

HTTP 클라이언트
(웹 브라우저)



```
body = "<html><body>Hello, my simple Web Server!</body> </html>"
```

4. 파이썬으로 간단한 HTTP 서버 제작하기

■ 간단한 HTTP 서버 프로그램

파일명: simpleHttpServer.py

```
1 # Only python으로 simple HTTP server 제작하기
2
3 import socket
4
5 IP = 'localhost'
6 PORT = 10000
7 address = (IP, PORT)
8
9 print("++ Simple HTTP server")
10 # TCP 소켓 생성
11 server = socket.socket()
12 # (부가코드) wiError 10048 에러해결을 위한 코드
13 server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
14 # bind()
15 server.bind(address)
16 # listen()
17 server.listen()
18 print("++ HTTP Server wait.....")
19
```

TCP 소켓 프로그램과 동일

HTTP
TCP
IP
MAC

4. 파이썬으로 간단한 HTTP 서버 제작하기

■ HTTP 서버 프로그램

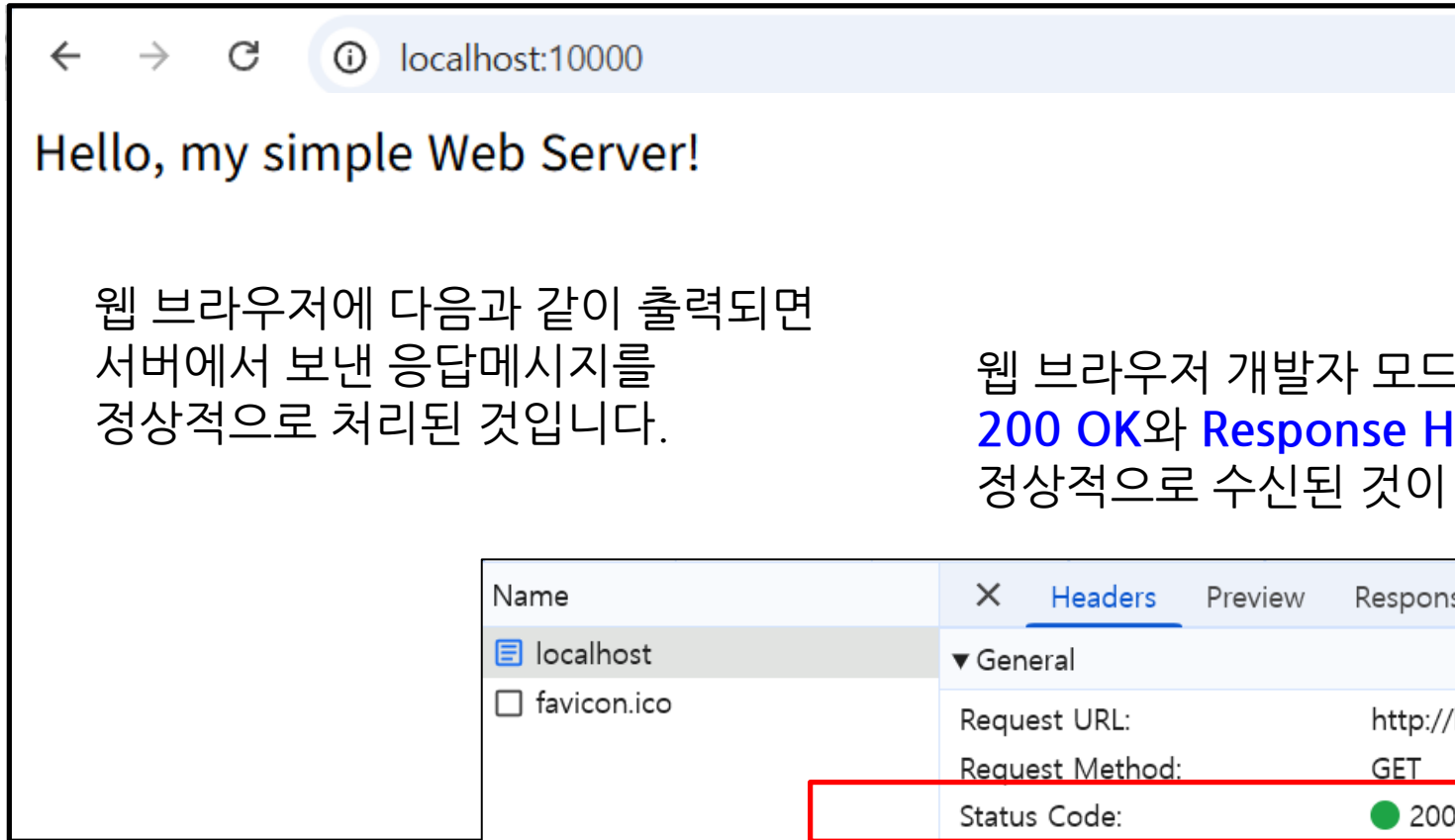
- 웹 브라우저로 요청메시지 받고, 응답메시지 보내기

```
19 while True:
20     # accept()
21     conn, addr = server.accept()
22     print(f"{addr} is connected")
23     # 웹브라우저 요청메시지 수신
24     req_message = conn.recv(1024).decode('utf-8')
25     print("Request message received")
26     lines = req_message.splitlines() # 줄단위로 분리
27     for line in lines:
28         print(line)
29     # HTTP 응답메시지 보내기
30     # 응답메시지 구조 = status_line + headers + CRLF(\r\n) + body
31     body = "<html><body>Hello, my simple HTTP Server!</body></html>"
32     status_line = 
33     headers = 
34     CRLF = "\r\n"
35     res_message = status_line + headers + CRLF + body
36     conn.sendall(res_message.encode('utf-8'))
37     print("++ Response message Send")
38     conn.close()
39 server.close()
40 print("++ HTTP Server End")
```

*HTTP 응답 메시지를 만들어 전송해 보세요.
(body는 제시되어 있음)*

4. 파이썬으로 간단한 HTTP 서버 제작하기

■ 웹 브라우저에서 서버에 접속해 보세요



웹 브라우저 개발자 모드(F12)에서 **200 OK**와 **Response Headers** 정보가 정상적으로 수신된 것이 보입니다.

Name	×	Headers	Preview	Response	Initiator	Timing
localhost		▼ General				
☐ favicon.ico		Request URL:	http://localhost:10000/			
		Request Method:	GET			
		Status Code:	● 200 OK			
		Remote Address:	127.0.0.1:10000			
		Referrer Policy:	strict-origin-when-cross-origin			
		▼ Response Headers	☐ Raw			
		Content-Length:	54			
		Content-Type:	text/html			

4. 파이썬으로 간단한 HTTP 서버 제작하기

■ 정상출력이 안되는 경우



사이트에 연결할 수 없음

연결이 재설정되었습니다.

다음 방법을 시도해 보세요.

- 연결 확인
- 프록시 및 방화벽 확인
- Windows 네트워크 문제 해결

ERR_CONNECTION_RESET

웹 브라우저 개발자 모드(F12)에서
다음을 확인해 보세요

- 1) 200 OK가 수신되었는지?
- 2) 응답 헤더가 제대로 수신되었는지?

그렇지 않다면 서버의 응답메시지 작성에
문제가 있다는 것입니다.

Name	✕	Headers	Preview	Response	Initiator	Timing	Cookies
✕ localhost	▼	General					
data:image/png;base...		Request URL:		http://localhost:10000/			
data:image/png;base...		Referrer Policy:		strict-origin-when-cross-origin			
data:image/png;base...		▶ Response Headers (0)					
✕ localhost	▼	Request Headers					
✕ localhost		Accept:		text/html,application/xhtml+xml,application/			
✕ localhost							

참고로 헤더가 제대로 인식되지 않은 경우

9주차 과제. 파이썬으로 간단한 HTTP 서버 제작하기

■ 과제명: 파이썬으로 간단한 HTTP 서버 제작하기

- 제출방법: 9주차 실습내용 중 '파이썬으로 간단한 HTTP 서버제작' 파이썬 파일을 과제 제출란에 제출
- 첨부파일: `simpleHttpServer_이름.py`
- 프로그램 상 오류 혹은 이해가 안 되는 부분은 comment 남겨 주세요.



Thank You
